

Library Management System

Evaluated Course Project Report

Course: Programming in Java

Platform: VITyarthi — Flipped Course Evaluation

LUKE NOBLE

24BEC10025

10-09-26

1. Introduction

This report documents the design, architecture, and implementation of a command-line **Library Management System** developed in core Java as the Evaluated Course Project for the **Programming in Java** course. The project applies object-oriented programming principles, layered software architecture, file-based persistence, and structured exception handling to build a working, terminal-executable application that a librarian could realistically use to manage members, book inventory, and borrowing activity.

2. Problem Statement

Small institutional libraries frequently manage inventories, members, and borrowing records manually, using registers or spreadsheets. This approach is error-prone, hard to search, and makes tracking overdue books and fines difficult. The project addresses this by providing a lightweight, dependency-free desktop application that a librarian can run from any terminal to manage members, track book inventory, and handle the issue/return workflow — including automatic fine calculation for overdue returns — without needing to set up a database server.

3. Functional Requirements

3.1 Member Management

- Add, update, and remove members, with email format validation.
- Search members by name (partial, case-insensitive match).
- Track and record payment of outstanding fines.

3.2 Book Inventory Management

- Add, update, and remove books (full CRUD).
- Track total copies vs. currently available copies.
- Search books by title (partial, case-insensitive match).

3.3 Issue / Return & Fine Calculation

- Issue a book to a member, subject to availability and the member having no unpaid fines.
- Return a book, with automatic fine calculation for late returns (Rs.5/day beyond a 14-day loan period).
- View a specific member's currently active (unreturned) loans.

3.4 Reporting

- View the full list of transactions (issued and returned).
- View currently overdue books across all members.

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	In-memory HashMaps/Lists for O(1)-O(n) lookups; disk writes only occur on state-changing operations, not on reads
Reliability	Every service method validates business rules (e.g. availability, duplicate IDs) before mutating in-memory state
Usability	Simple numbered console menus with looping submenus, clear input prompts, and human-readable [ERROR] messages
Maintainability	Strict layered architecture (UI / Service / Storage / Model) means each concern can be modified independently
Error Handling	A custom checked exception (LibraryException) is used throughout the service layer to represent domain errors
Scalability	The storage layer is abstracted behind a small, focused class (FileStorage), so it can be replaced with JDBC/Redis

5. System Architecture

The application follows a four-layer architecture that separates user interaction, business logic, persistence, and data modeling. This separation keeps each class focused on a single responsibility and makes the system easy to extend — for instance, the file-based storage layer could be replaced with a JDBC-backed implementation without any change to the service or UI layers.

System Architecture — Layered Design

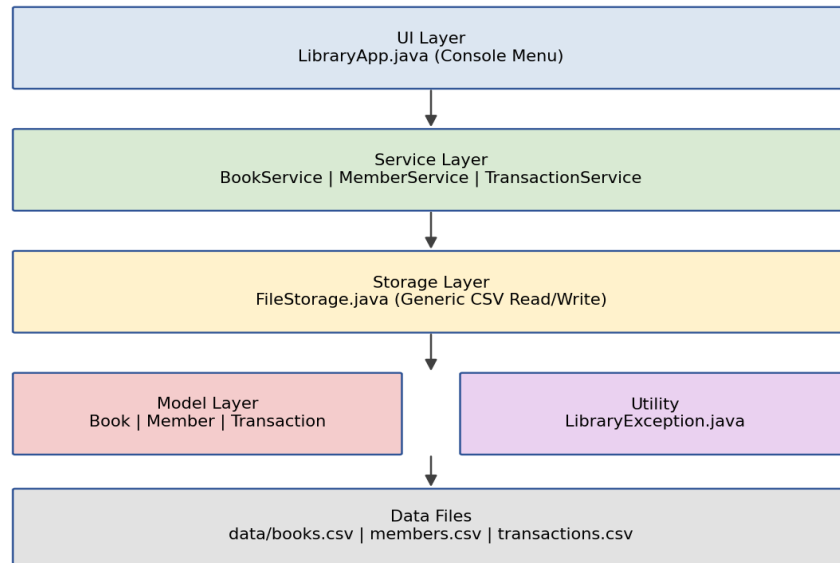


Figure 1: Layered system architecture.

6. Design Diagrams

6.1 Use Case Diagram

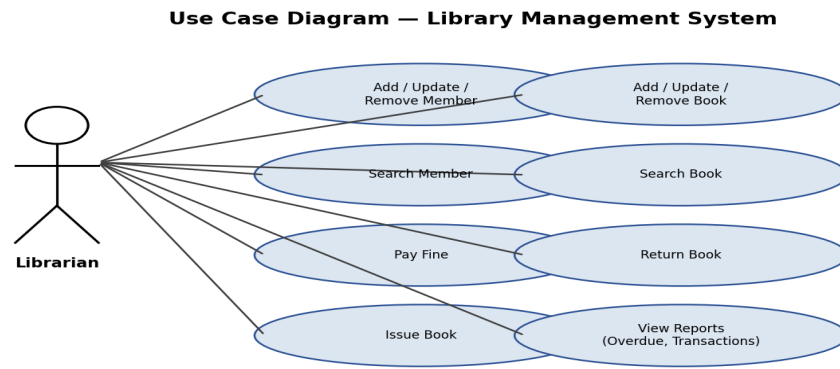


Figure 2: Use case diagram — Librarian's interactions with the system.

6.2 Process Flow / Workflow Diagram

Book Issue → Return Workflow

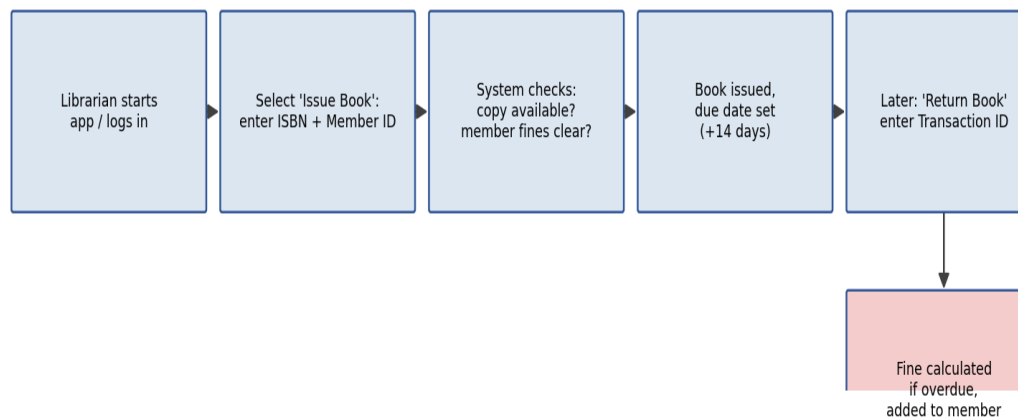


Figure 3: Book issue → return workflow, including overdue fine calculation.

6.3 Class Diagram (Simplified)

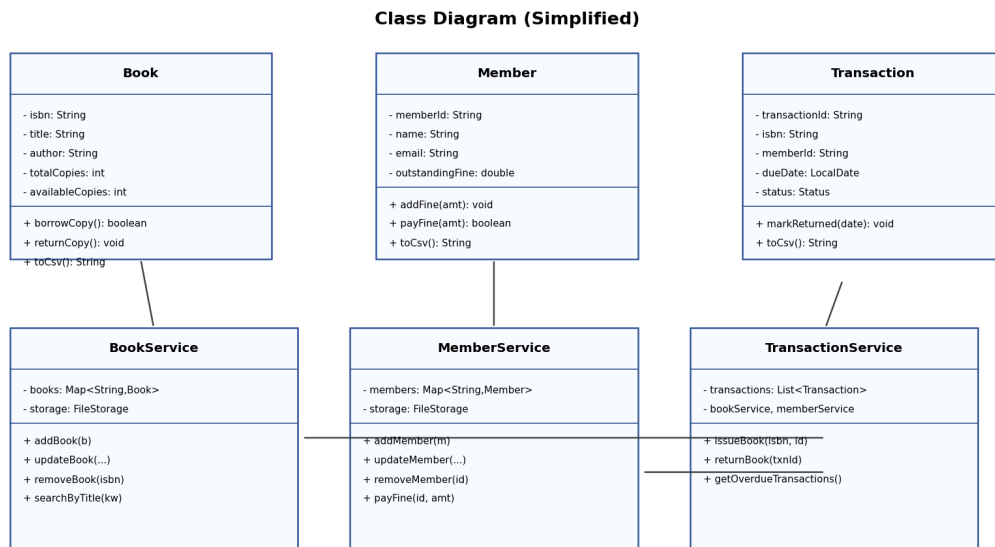


Figure 4: Simplified class diagram showing model and service layer classes.

6.4 Sequence Diagram — Issue Book

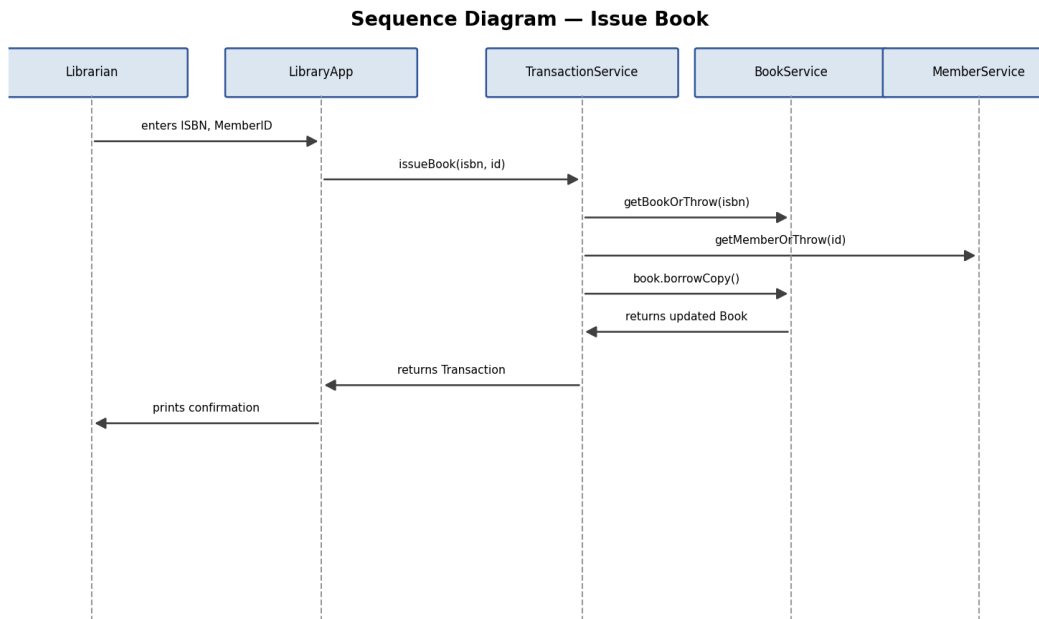


Figure 5: Sequence of calls when a librarian issues a book to a member.

6.5 Data Schema / ER Diagram

The application uses flat CSV files rather than a relational database, but the three entities still have a clear relational structure, shown below. **Book** and **Member** are independent entities; **Transaction** references both via their primary keys (ISBN and Member ID respectively), forming a many-to-many relationship between books and members mediated by transactions.

Data Schema (CSV-backed entities)

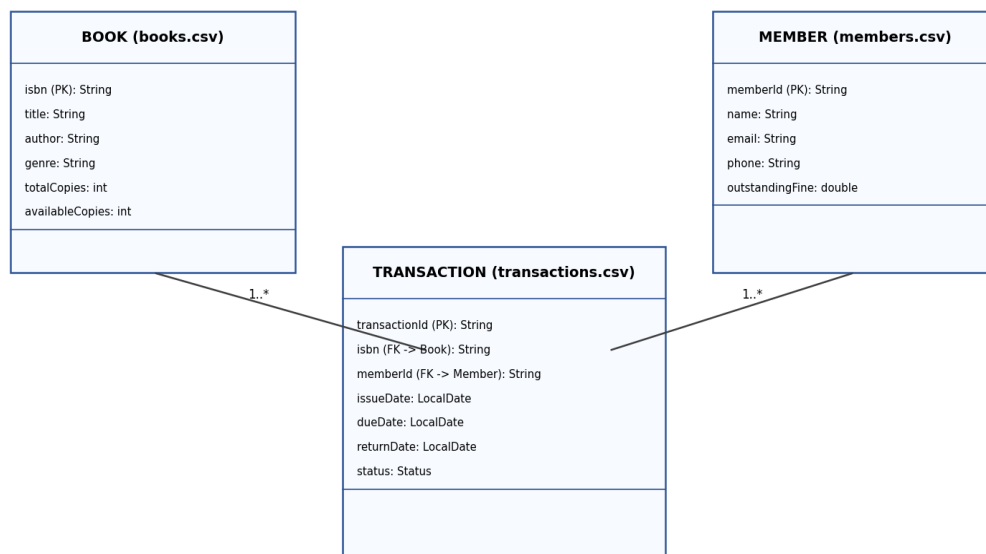


Figure 6: Data schema across the three CSV-backed entities.

7. Design Decisions & Rationale

- **File-based CSV storage instead of a database:** chosen to keep the project runnable with zero external setup (no DB server/driver install), satisfying the command-line-executability requirement while still demonstrating a clean storage abstraction that could be swapped for JDBC later.
- **Layered architecture (UI / Service / Storage / Model):** chosen over a single monolithic class so that business rules (e.g. fine calculation, availability checks) are testable independently of console I/O.
- **Custom checked exception (LibraryException):** chosen over generic RuntimeExceptions so that expected domain errors (duplicate ID, book unavailable, invalid email) are handled distinctly from genuine bugs, and so the compiler enforces that callers handle them.
- **In-memory maps backed by CSV persistence:** chosen for fast lookups during a session, with the full collection re-written to disk after each mutation to keep the file simple and avoid partial-write corruption.

8. Implementation Details

The project is organized into 9 Java source files across four packages:

Package	Class	Responsibility
model	Book, Member, Transaction	Domain entities with CSV (de)serialization logic.
service	BookService	CRUD and search operations for the book inventory.
service	MemberService	CRUD, search, and fine-payment operations for members.
service	TransactionService	Issue/return workflow, fine calculation, and reporting queries.
storage	FileStorage	Generic, reusable CSV read/write helper used by all three services.
util	LibraryException	Custom checked exception for domain-level errors.
app	LibraryApp	Console UI: menu loop, input handling, and top-level error display.

Key implementation details: overdue fines are calculated using `ChronoUnit.DAYS.between(dueDate, today)` multiplied by a fixed per-day rate; CSV fields that might contain commas (e.g. names) are escaped by substituting semicolons before writing and reversed on read; and the transaction ID counter is recovered on startup by scanning existing transaction IDs, so IDs remain unique across restarts.

9. Screenshots / Results

[Insert terminal screenshots here showing: (1) the main menu, (2) a member being added, (3) a book being issued, and (4) the overdue-books report. Run the application as described in the README and capture your terminal output.]

10. Testing Approach

Testing was performed through manual end-to-end scenario walkthroughs covering both the happy path and error conditions, executed directly against the compiled CLI:

- **Happy path:** add a member → add a book → issue the book → view it in the member's active loans → return it before the due date → confirm no fine was applied.
- **Overdue path:** issue a book, manually verify the fine calculation logic against a due date in the past, confirm the fine is added to the member's balance and blocks further issues until paid.
- **Validation/error handling:** attempted duplicate member IDs, invalid email formats, issuing an unknown ISBN, and returning an already-returned transaction — all correctly produced a clean [ERROR] message rather than crashing the application.
- **Persistence check:** restarted the application between operations to confirm data was correctly reloaded from the CSV files in `data/`.

11. Challenges Faced

- Designing a CSV escaping scheme that could safely handle commas in free-text fields (like names and titles) without needing a full CSV parsing library.
- Coordinating state changes across three services (Book, Member, Transaction) for a single logical operation (issuing/returning a book) while keeping each service's persistence self-contained.
- Getting submenu navigation right so that each menu loops on invalid/repeated actions instead of exiting back to the main menu after a single action, which is important for usability.

12. Learnings & Key Takeaways

- Practical experience structuring a Java application into clear layers (UI/service/storage/model) rather than one large class, and seeing how this simplifies reasoning about each part.
- Hands-on use of custom checked exceptions to separate expected domain errors from genuine bugs.
- Working with `java.time` (`LocalDate`, `ChronoUnit`) for real date arithmetic (due dates, overdue calculation).
- Designing a simple but real persistence layer without an external database, and understanding the trade-offs versus using JDBC/a relational database.

13. Future Enhancements

- Migrate storage from CSV files to an embedded database (e.g. SQLite) via JDBC.

- Add a JUnit test suite to automate the manual test scenarios described above.
- Add book reservation/waitlist support for popular titles.
- Add a thin REST API layer on top of the existing service layer to enable a future web/GUI front-end.

14. References

- Oracle Java SE 21 Documentation — docs.oracle.com/en/java/javase/21/
- Java java.time API Reference (LocalDate, ChronoUnit) — docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/package-summary.html
- Effective Java, 3rd Edition, Joshua Bloch — referenced for general Java design-practice conventions applied in this project.